

BÁO CÁO MÃ NGUỒN - LAB 01

****Sinh viên thực hiện:**** Lê Viết Đức

****MSSV:**** 23DTHA4 / 0533

****Môn học:**** Thực hành Bảo mật thông tin nâng cao

Đường dẫn file: `lab-01\ex02\_01.py`

```
ten = input("Nhập tên của bạn: ")
tuoi = input("Nhập tuổi của bạn: ")
print("Chào mừng,", ten, "! Bạn", tuoi, "tuổi.")
```

Đường dẫn file: `lab-01\ex02\_02.py`

```
ban_kinh = float(input("Nhập bán kính của hình tròn: "))
dien_tich = 3.14 * (ban_kinh ** 2)
print("Diện tích của hình tròn là:", dien_tich)
```

Đường dẫn file: `lab-01\ex02\_03.py`

```
so = int(input("Nhập một số nguyên: "))
if so % 2 == 0:
    print(so, "là số chẵn.")
else:
    print(so, "là số lẻ.")
```

Đường dẫn file: `lab-01\ex02\_04.py`

```
j = []
for i in range(2000, 3201):
    if (i % 7 == 0) and (i % 5 != 0):
        j.append(str(i))
print('.'.join(j))
```

Đường dẫn file: `lab-01\ex02\_05.py`

```
so_gio_lam = float(input("Nhập số giờ làm việc mỗi tuần: "))
luong_gio = float(input("Nhập thù lao trên mỗi giờ làm việc tiêu chuẩn: "))
gio_tieu_chuan = 44

if so_gio_lam <= gio_tieu_chuan:
    tong_luong = so_gio_lam * luong_gio
else:
    gio_vuot_chuan = so_gio_lam - gio_tieu_chuan
    tong_luong = (gio_tieu_chuan * luong_gio) + (gio_vuot_chuan * luong_gio * 1.5)

print(f"Số tiền thực nhận của nhân viên: {tong_luong}")
```

Đường dẫn file: `lab-01\ex02\_06.py`

```
input_str = input("Nhập X, Y: ")
dimensions = [int(x) for x in input_str.split(',')]
rowNum = dimensions[0]
colNum = dimensions[1]
multilist = [[0 for col in range(colNum)] for row in range(rowNum)]

for row in range(rowNum):
    for col in range(colNum):
        multilist[row][col] = row * col

print(multilist)
```

Đường dẫn file: `lab-01\ex02\_07.py`

```

print("Nhập các dòng văn bản (Nhập 'done' để kết thúc):")
lines = []
while True:
    line = input()
    if line.lower() == 'done':
        break
    lines.append(line)

print("\nCác dòng chữ in hoa:")
for line in lines:
    print(line.upper())

```

Đường dẫn file: `lab-01\ex02\_08.py`

```

def chia_het_cho_5(so_nhi_phan):
    so_thap_phan = int(so_nhi_phan, 2)
    return so_thap_phan % 5 == 0

chuoi_nhi_phan = input("Nhập chuỗi số nhị phân (phân cách bởi dấu phẩy): ")
cac_so_nhi_phan = chuoi_nhi_phan.split(',')
so_chia_het_cho_5 = [so for so in cac_so_nhi_phan if chia_het_cho_5(so)]

if len(so_chia_het_cho_5) > 0:
    print("Các số nhị phân chia hết cho 5 là:", ', '.join(so_chia_het_cho_5))
else:
    print("Không có số nhị phân nào chia hết cho 5.")

```

Đường dẫn file: `lab-01\ex02\_09.py`

```

def kiem_tra_so_nguyen_to(n):
    if n <= 1:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

number = int(input("Nhập vào số cần kiểm tra: "))
if kiem_tra_so_nguyen_to(number):
    print(number, "là số nguyên tố.")
else:

```

```
print(number, "không phải là số nguyên tố.")
```

Đường dẫn file: `lab-01\ex02\_10.py`

```
def dao_nguoc_chuoi(chuoi):
    return chuoi[::-1]

input_string = input("Nhập chuỗi cần đảo ngược: ")
print("Chuỗi đảo ngược là:", dao_nguoc_chuoi(input_string))
```

Đường dẫn file: `lab-01\ex03\ex03\_01.py`

```
# Nhập danh sách từ người dùng
input_list = input("Nhập danh sách các số, cách nhau bằng dấu phẩy: ")
numbers = [int(x) for x in input_list.split(',')]
sum_even = sum(x for x in numbers if x % 2 == 0)
print("Tổng các số chẵn trong danh sách là:", sum_even)
```

Đường dẫn file: `lab-01\ex03\ex03\_02.py`

```
input_list = input("Nhập danh sách các số, cách nhau bằng dấu phẩy: ")
numbers = [int(x) for x in input_list.split(',')]
numbers.reverse()
print("Danh sách sau khi đảo ngược:", numbers)
```

Đường dẫn file: `lab-01\ex03\ex03\_03.py`

```
input_list = input("Nhập danh sách các số, cách nhau bằng dấu phẩy: ")
numbers = [int(x) for x in input_list.split(',')]
my_tuple = tuple(numbers)
print("List:", numbers)
```

```
print("Tuple tạo từ List:", my_tuple)
```

Đường dẫn file: `lab-01\ex03\ex03\_04.py`

```
input_tuple = input("Nhập các phần tử của tuple, cách nhau bằng dấu phẩy: ")
my_tuple = tuple(input_tuple.split(','))
print("Tuple:", my_tuple)
print("Phần tử đầu tiên:", my_tuple[0])
print("Phần tử cuối cùng:", my_tuple[-1])
```

Đường dẫn file: `lab-01\ex03\ex03\_05.py`

```
input_list = input("Nhập danh sách các phần tử, cách nhau bằng dấu phẩy: ")
elements = input_list.split(',')
frequency = {}
for item in elements:
    item = item.strip()
    frequency[item] = frequency.get(item, 0) + 1
print("Số lần xuất hiện của các phần tử:", frequency)
```

Đường dẫn file: `lab-01\ex03\ex03\_06.py`

```
my_dict = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
print("Dictionary ban đầu:", my_dict)
key_to_delete = input("Nhập khóa cần xóa: ")
if key_to_delete in my_dict:
    del my_dict[key_to_delete]
    print("Dictionary sau khi xóa phần tử:", my_dict)
else:
    print("Khóa không tồn tại trong dictionary.")
```

Đường dẫn file: `lab-01\ex04\main.py`

```

from student_manager import StudentManager

def show_student_list(students):
    if not students:
        print("\nDanh sách sinh viên trống.")
        return
    print("\n" + "-" * 85)
    print(f"{'ID':<5} | {'Họ và tên':<25} | {'Giới tính':<10} | {'Chuyên ngành':<20} | {'ĐTB':<5} | {'Học lực':<5}")
    print("-" * 85)
    for s in students:
        print(f"{s.id:<5} | {s.name:<25} | {s.gender:<10} | {s.major:<20} | {s.gpa:<5.1f} | {s.rank:<5}")
        print("-" * 85)

def main():
    manager = StudentManager()

    # Seed some sample data for demonstration
    manager.add_student("Nguyen Van A", "Nam", "ATTT", 8.5)
    manager.add_student("Tran Thi B", "Nu", "CNTT", 6.8)
    manager.add_student("Le Van C", "Nam", "KTPM", 4.5)

    while True:
        print("\n=== CHƯƠNG TRÌNH QUẢN LÝ SINH VIÊN ===")
        print("1. Thêm sinh viên")
        print("2. Cập nhật thông tin sinh viên bởi ID")
        print("3. Xóa sinh viên bởi ID")
        print("4. Tìm kiếm sinh viên theo tên")
        print("5. Sắp xếp sinh viên theo Điểm trung bình (GPA)")
        print("6. Sắp xếp sinh viên theo Chuyên ngành (Major)")
        print("7. Hiển thị danh sách sinh viên")
        print("8. Thoát")

        choice = input("Nhập lựa chọn của bạn (1-8): ").strip()

        if choice == '1':
            print("\n--- THÊM SINH VIÊN MỚI ---")
            name = input("Nhập tên sinh viên: ")
            gender = input("Nhập giới tính: ")
            major = input("Nhập chuyên ngành: ")
            try:
                gpa = float(input("Nhập điểm trung bình (GPA): "))
                if gpa < 0 or gpa > 10:
                    print("Lỗi: Điểm GPA phải nằm trong khoảng từ 0 đến 10.")
                    continue
                manager.add_student(name, gender, major, gpa)
                print("Thêm sinh viên thành công!")
            except ValueError:
                print("Lỗi: Điểm GPA phải là một số thực.")

```

```

elif choice == '2':
    print("\n--- CẬP NHẬT THÔNG TIN SINH VIÊN ---")
    try:
        student_id = int(input("Nhập ID sinh viên cần cập nhật: "))
        name = input("Nhập tên mới: ")
        gender = input("Nhập giới tính mới: ")
        major = input("Nhập chuyên ngành mới: ")
        gpa = float(input("Nhập điểm GPA mới: "))
        if gpa < 0 or gpa > 10:
            print("Lỗi: Điểm GPA phải nằm trong khoảng từ 0 đến 10.")
            continue
        if manager.update_student(student_id, name, gender, major, gpa):
            print("Cập nhật thông tin sinh viên thành công!")
        else:
            print(f"Không tìm thấy sinh viên với ID = {student_id}")
    except ValueError:
        print("Lỗi: ID phải là số nguyên và GPA phải là số thực.")

elif choice == '3':
    print("\n--- XÓA SINH VIÊN ---")
    try:
        student_id = int(input("Nhập ID sinh viên cần xóa: "))
        if manager.delete_student(student_id):
            print("Xóa sinh viên thành công!")
        else:
            print(f"Không tìm thấy sinh viên với ID = {student_id}")
    except ValueError:
        print("Lỗi: ID phải là số nguyên.")

elif choice == '4':
    print("\n--- TÌM KIẾM SINH VIÊN THEO TÊN ---")
    name = input("Nhập tên cần tìm kiếm: ")
    results = manager.search_by_name(name)
    show_student_list(results)

elif choice == '5':
    print("\n--- SẮP XẾP SINH VIÊN THEO GPA (GIẢM DẦN) ---")
    manager.sort_by_gpa()
    show_student_list(manager.get_all_students())

elif choice == '6':
    print("\n--- SẮP XẾP SINH VIÊN THEO CHUYÊN NGÀNH ---")
    manager.sort_by_major()
    show_student_list(manager.get_all_students())

elif choice == '7':
    print("\n--- DANH SÁCH SINH VIÊN ---")
    show_student_list(manager.get_all_students())

elif choice == '8':

```

```

        print("\nTạm biệt!")
        break

    else:
        print("Lựa chọn không hợp lệ. Vui lòng nhập lại từ 1 đến 8.")

if __name__ == '__main__':
    main()

```

Đường dẫn file: `lab-01\ex04\student.py`

```

class Student:
    def __init__(self, student_id, name, gender, major, gpa):
        self.id = student_id
        self.name = name
        self.gender = gender
        self.major = major
        self.gpa = gpa
        self.rank = self.calculate_rank()

    def calculate_rank(self):
        if self.gpa >= 8.0:
            return "Giỏi"
        elif self.gpa >= 6.5:
            return "Khá"
        elif self.gpa >= 5.0:
            return "Trung bình"
        else:
            return "Yếu"

```

Đường dẫn file: `lab-01\ex04\student\_manager.py`

```

from student import Student

class StudentManager:
    def __init__(self):
        self.students = []
        self.next_id = 1

    def add_student(self, name, gender, major, gpa):
        student = Student(self.next_id, name, gender, major, gpa)
        self.students.append(student)

```



```
        self.next_id += 1
        return student

def update_student(self, student_id, name, gender, major, gpa):
    for s in self.students:
        if s.id == student_id:
            s.name = name
            s.gender = gender
            s.major = major
            s.gpa = gpa
            s.rank = s.calculate_rank()
            return True
    return False

def delete_student(self, student_id):
    for i, s in enumerate(self.students):
        if s.id == student_id:
            del self.students[i]
            return True
    return False

def search_by_name(self, name):
    results = []
    for s in self.students:
        if name.lower() in s.name.lower():
            results.append(s)
    return results

def sort_by_gpa(self):
    self.students.sort(key=lambda x: x.gpa, reverse=True)

def sort_by_major(self):
    self.students.sort(key=lambda x: x.major.lower())

def get_all_students(self):
    return self.students
```